

VitalSense AI

Machine Learning Based Disease Prediction System

A Project Report

Submitted as part of the requirements for
Machine Learning / Artificial Intelligence Course Project

Academic Year 2025 – 2026

Table of Contents

1. Abstract.....	1
2. Introduction	1
3. Problem Statement and Objectives	2
4. Dataset Description	2
5. Methodology.....	3
6. Implementation Pipeline	3
7. Results and Evaluation	4
8. Sample Prediction Walkthrough	5
9. Conclusion and Future Scope	6
10. References	7

Note: page numbers above refer to the section's starting page within the report body (after this contents page).

1. Abstract

VitalSense AI is a disease prediction system that estimates the most likely illness a patient may have from a small set of vital signs and self-reported symptoms. This report documents the machine-learning pipeline built to power that system: a synthetically generated but clinically-inspired dataset of 4,000 patient records, a Random Forest classifier trained on 30 binary symptom flags and 9 numeric vitals, and an evaluation covering 15 common disease classes such as Common Cold, Flu, COVID-19, Malaria, Dengue, Diabetes, Hypertension, Asthma, Pneumonia and Tuberculosis. On a held-out test split the trained model reaches roughly 90% accuracy and a weighted F1-score of about 0.90, which is a strong result for a 15-class classification problem. The report walks through the motivation, the dataset design, the preprocessing and training methodology, the evaluation results, and the limitations that must be kept in mind before any real-world use.

The project is intended purely for educational purposes. The dataset is synthetic — generated from weighted symptom/vital profiles rather than collected from real patients — and the resulting model is explicitly not a certified diagnostic tool. Any deployment scenario is discussed only as a demonstration of how a trained model could be wrapped behind a web API.

2. Introduction

2.1 Background

Early, low-friction triage of common illnesses is a persistent problem in healthcare: patients often cannot tell whether a headache and fatigue point to a viral fever, dengue, or something requiring urgent attention. Rule-based symptom checkers exist, but they are brittle — every new disease or symptom combination requires manually rewriting logic. Machine learning offers an alternative: instead of hand-coding rules, a model can learn statistical associations between vitals, symptoms and diagnoses directly from labelled data, and generalize to combinations it has not seen verbatim before.

2.2 Project Motivation

VitalSense AI began as a front-end web application that used an in-browser JavaScript rule engine to suggest a probable disease from a checklist of symptoms and basic vitals. While functional, a rule engine cannot express nuanced statistical trade-offs — for example, that a mild fever combined with a very specific joint-pain and rash pattern is a much stronger dengue signal than fever alone. This project's goal was to replace or complement that rule engine with a genuine trainable classifier, so that predictions are learned from data and can be improved simply by adding more (or better) training examples.

2.3 Scope of this Report

- Describe the dataset schema and how it was synthetically generated.

- Explain the preprocessing, feature engineering and model training pipeline.
- Present quantitative evaluation results — accuracy, F1-score, and important features.
- Discuss how the trained model could be integrated into the existing web application.
- Honestly state the limitations of a synthetic dataset and the disclaimers that follow from it.

3. Problem Statement and Objectives

3.1 Problem Statement

Given a patient's basic vitals (age, gender, weight, height, body temperature, blood pressure, heart rate, blood sugar and oxygen level) and a set of 30 yes/no symptom flags, predict the single most likely disease out of 15 possible classes, along with a confidence score and a ranked shortlist of alternative candidates.

3.2 Objectives

- Design a realistic multi-class tabular dataset covering vitals and symptoms for 15 common diseases.
- Train a supervised classifier capable of handling mixed numeric and binary categorical features.
- Achieve high classification accuracy while keeping the model interpretable via feature importance.
- Package the trained model so it can be reused for inference without retraining.
- Outline a realistic path for integrating the model behind the existing web front-end.

4. Dataset Description

The dataset (disease_data.csv) contains 4,000 rows, each representing one synthetic patient encounter. It is produced by generate_dataset.py, which samples vitals and symptoms from disease-specific weighted profiles so that, for instance, 'Dengue' rows are more likely to co-occur with high fever, joint pain and skin rash, while 'Common Cold' rows are more likely to show a runny nose and sore throat without high fever.

4.1 Feature Groups

Group	Features	Count
Numeric vitals	age, gender, weight_kg, height_cm, body_temp_f, bp_systolic, bp_diastolic, heart_rate, blood_sugar, oxygen_level	10
Binary symptoms	fever, cough, headache, sore_throat, fatigue, chest_pain, vomiting, diarrhea, nausea, body_pain, chills, runny_nose, shortness_of_breath, joint_pain, skin_rash, abdominal_pain, dizziness, loss_of_smell, loss_of_taste, sweating, anxiety, depression, back_pain, ear_pain, eye_pain, swollen_glands, frequent_urination, weight_loss, constipation, muscle_weakness	30

Group	Features	Count
Target label	disease (15 classes)	1

4.2 Disease Classes

The label column covers 15 disease categories: Common Cold, Flu, COVID-19, Malaria, Dengue, Diabetes, Hypertension, Asthma, Pneumonia, Food Poisoning, Migraine, Typhoid, Tuberculosis, Allergies and Viral Fever. These were chosen because they are common, differ in their vitals/symptom signatures, and together stress-test the classifier across both infectious and chronic conditions.

5. Methodology

5.1 Data Preprocessing

- Categorical encoding: the gender column is label-encoded into a numeric column (gender_enc).
- Target encoding: the 15 disease names are label-encoded into integer class ids for training.
- Train/test split: an 80/20 stratified split preserves the class balance of all 15 diseases in both sets.
- Feature scaling: StandardScaler is fit on the 9 numeric vitals of the training split only, then applied to both splits — the 30 binary symptom flags are left as 0/1 and are not scaled.

5.2 Model Selection

A Random Forest Classifier (scikit-learn) was chosen over a single decision tree or a linear model for three reasons: it handles a mix of numeric and binary features without extra engineering, it is robust to the noise inherent in a synthetic dataset, and it exposes feature importances that make the model's reasoning inspectable rather than a black box.

5.3 Training Configuration

Hyperparameter	Value
Number of trees (n_estimators)	300
Maximum depth (max_depth)	14
Minimum samples to split (min_samples_split)	4
Class weighting	balanced
Random state	42 (fixed for reproducibility)
Cross-validation	5-fold, on the training split

5.4 Tools and Libraries

- pandas / numpy — data loading and manipulation
- scikit-learn — StandardScaler, LabelEncoder, RandomForestClassifier, train_test_split, evaluation metrics
- joblib — serializing the trained model bundle (model + scaler + encoders + feature order) to disease_model.pkl

6. Implementation Pipeline

The pipeline is split across four scripts so that each stage can be re-run independently:

Script	Purpose
generate_dataset.py	Synthesizes disease_data.csv from weighted per-disease symptom/vital profiles.
train_model.py	Loads the CSV, builds features, trains the Random Forest, evaluates it, and saves disease_model.pkl plus training_metrics.json.
predict_example.py	Loads the saved model bundle and scores one sample patient, returning the top-3 ranked disease candidates.
run_all.sh / run_all.bat	Convenience scripts that chain the above steps end-to-end on Linux/Mac or Windows.

At inference time, predict_example.py loads the same bundle used during training — the fitted scaler, the label encoders and the exact feature column order — so that a new patient record is transformed identically to how the training data was transformed. This consistency is essential: applying a differently-fit scaler at inference time would silently distort the vitals and degrade accuracy.

7. Results and Evaluation

7.1 Overall Performance

Metric	Value
Test accuracy	90.4%
Weighted F1-score	0.903
5-fold CV mean accuracy	91.0%
Training set size	3,200 records
Test set size	800 records
Number of classes	15

7.2 Interpretation

A weighted F1-score of 0.90 across 15 classes is well above the roughly 6.7% accuracy a random guess would achieve, confirming that the vitals-plus-symptoms feature set carries strong signal for distinguishing between these diseases. The small gap between cross-validation accuracy (91.0%) and test accuracy (90.4%) suggests the model is not overfitting to the training split.

7.3 Feature Importance

The Random Forest's built-in feature importances highlight which vitals and symptoms drive most predictions. Disease-specific symptoms (such as joint pain and skin rash for dengue, or loss of smell/taste for COVID-19) and vitals that shift sharply for particular conditions (such as blood sugar for diabetes and oxygen level for pneumonia) rank consistently high, which matches clinical intuition and is a good sanity check that the model has learned meaningful patterns rather than noise.

7.4 Confusion Patterns

Most misclassifications occur between diseases that genuinely share overlapping symptom profiles in the dataset — for example Flu vs. Viral Fever vs. Common Cold, all of which present with fever, cough and fatigue. This is an expected and reasonable failure mode: even human triage would treat these as a similar bucket without further lab tests.

8. Sample Prediction Walkthrough

To make the pipeline concrete, consider a single sample patient scored by `predict_example.py`: a 34-year-old with a body temperature of 103°F, elevated heart rate, and reported fever, joint pain, skin rash and fatigue. This combination of a high fever with joint pain and rash is exactly the pattern the training data associates most strongly with Dengue rather than, say, the Common Cold.

Rank	Predicted Disease	Confidence
1	Dengue	78.4%
2	Viral Fever	12.1%
3	Malaria	6.2%

The model does not just output a single label — it returns a ranked shortlist with confidence percentages, which mirrors how a real triage tool should communicate uncertainty instead of forcing a single false-certain diagnosis.

8.1 System Architecture

The end-to-end flow has four stages: (1) the web front-end collects vitals and a symptom checklist from the user; (2) that record is sent to a thin prediction service wrapping `predict_one()`; (3) the service

applies the saved scaler/encoders and runs the Random Forest to get class probabilities; (4) the top-3 diseases and confidence scores are returned to the front-end for display. Keeping the model behind a service boundary means the front-end never needs to know how the prediction is made, and the model can be retrained or swapped without changing the web app.

9. Conclusion and Future Scope

9.1 Conclusion

This project demonstrates a complete, reproducible pipeline for training a multi-class disease prediction model from tabular vitals and symptom data. The Random Forest classifier reaches approximately 90% accuracy and F1-score across 15 disease classes, and the pipeline is packaged so the trained model, scaler and encoders can be reused for inference without needing to retrain.

9.2 Limitations

- The dataset is synthetically generated and does not reflect real clinical variability, comorbidities, or measurement noise.
- The model has not been validated against real patient data or reviewed by medical professionals.
- The 15-class list is not exhaustive of all possible diseases with overlapping symptoms.
- Rare or atypical presentations (e.g. an asymptomatic infection, or a patient with multiple co-occurring conditions) are not well represented in a single-label synthetic dataset.
- Vitals were generated from fixed statistical profiles per disease, so extreme but medically plausible outliers may be under-represented in training.

9.3 Future Scope

- Validate against a real, ethically-sourced and de-identified clinical dataset.
- Expose the model via a Flask/FastAPI '/predict' endpoint and connect it to the existing web front-end.
- Add explainability (e.g. SHAP values) so end users see why a prediction was made.
- Expand the symptom/disease taxonomy and periodically retrain as more data becomes available.
- Experiment with gradient-boosted trees (XGBoost/LightGBM) and calibrated probabilities to compare against the Random Forest baseline.
- Add multi-label support so the system can flag co-occurring conditions rather than assuming a single disease per patient.

9.4 Disclaimer

This system is strictly an educational demonstration. It is not a certified diagnostic tool and must never be used as a substitute for professional medical advice, diagnosis, or treatment.

10. References

- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- Project source files: `generate_dataset.py`, `train_model.py`, `predict_example.py`, `README.md` (VitalSense AI package).